



UNIVERSIDAD ABIERTA PARA ADULTOS

Escuela de Ingeniería y Tecnología

INFORME ASIGNACION ESCRITA II

Temática:

Uso de Listas Simplemente Enlazadas y Doblemente Enlazadas en la Solución de Problemas Y Complejidad Temporal Algorítmica.

DATOS DEL EQUIPO	
Equipo	Naranja - Equipo #15
Integrantes Y Matriculas	Hugo E. Montero Fulcar Matricula 100082929
	Erasmus José Minaya Taveras Matricula 100076710
	Albert Daniel Montero Ramírez Matricula 100078731
	Luis Ángel Mora Taveras Matricula 100076432
	Fernando Enrique Mesón Acosta Matricula 100027782
Asignatura	Estructura de Datos
Institución	Universidad Abierta para Adultos (UAPA)
Fecha	22 de mayo 2026

1. Introducción

El presente informe tiene como objetivo documentar la implementación de estructuras de datos dinámicas en el lenguaje de programación C++, aplicando el Paradigma Orientado a Objetos (POO). Específicamente, se abordan dos tipos de listas enlazadas: la lista simplemente enlazada y la lista doblemente enlazada.

Las listas enlazadas son estructuras de datos lineales donde cada elemento (nodo) contiene un valor y uno o más punteros que apuntan al nodo siguiente, o al siguiente y anterior en el caso de la lista doble. A diferencia de los arreglos, permiten una gestión dinámica de la memoria, lo que facilita la inserción y eliminación eficiente de elementos.

Como parte del Equipo Naranja (Equipo #15), se resolvieron los siguientes problemas:

- Ejercicio 1 (Lista Simple): Listado dinámico de tareas priorizadas.
- Ejercicio 2 (Lista Simple): Control de stock mínimo de productos.
- Ejercicio Común (Lista Doble): Navegación de historial web.

Cada solución incluye el análisis de complejidad temporal de cada función y la medición del tiempo real de ejecución utilizando la librería <chrono> de C++.

2. Marco Teórico

2.1 Lista Simplemente Enlazada

Una lista simplemente enlazada es una estructura de datos compuesta por nodos, donde cada nodo almacena un dato y un puntero al siguiente nodo de la secuencia. El ultimo nodo apunta a nullptr, indicando el fin de la lista. El acceso a los elementos es secuencial, comenzando desde la cabeza.

Características principales:

- Inserción al inicio: $O(1)$ - no requiere recorrer la lista.
- Inserción al final: $O(n)$ - debe recorrerse hasta el último nodo (sin puntero a cola).
- Inserción ordenada: $O(n)$ - recorre hasta encontrar la posición correcta.
- Búsqueda / acceso: $O(n)$ - en el peor caso recorre todos los nodos.
- Eliminación: $O(n)$ - requiere encontrar el nodo anterior al que se elimina.

2.2 Lista Doblemente Enlazada

Una lista doblemente enlazada extiende la lista simple agregando un segundo puntero en cada nodo que apunta al nodo anterior. Esto permite la navegación en ambas direcciones (hacia adelante y hacia atrás) y facilita operaciones como la eliminación directa de un nodo conocido.

Ventajas sobre la lista simple:

- Navegación bidireccional en $O(1)$.
- Eliminación de un nodo conocido en $O(1)$ (si se tiene el puntero directo al nodo).
- Inserción al final en $O(1)$ si se mantiene un puntero a la cola.
- Recorrido inverso en $O(n)$.

2.3 Complejidad Temporal (Notación Big-O)

La complejidad temporal mide como crece el tiempo de ejecución de un algoritmo en función del tamaño de la entrada (n). Las notaciones más comunes son:

Notación	Descripción
$O(1)$	Tiempo constante. No depende del tamaño de la entrada.
$O(\log n)$	Tiempo logarítmico. Crece lentamente al aumentar n .
$O(n)$	Tiempo lineal. Crece proporcionalmente con n .
$O(n^2)$	Tiempo cuadrático. Doble bucle sobre todos los elementos.

3. Ejercicio 1 - Listado Dinámico de Tareas Priorizadas



ejercicio1_tareas_priorizadas.cpp

3.1 Descripción del Problema

Una aplicación de gestión de tareas necesita mantener un listado de actividades pendientes ordenadas por prioridad. Cada nodo representa una tarea con: nombre, nivel de prioridad (entero) y estado (pendiente o completada).

El sistema debe:

- a) Insertar las tareas en orden descendente según su prioridad.
- b) Mostrar todas las tareas que estén en estado pendiente.
- c) Eliminar las tareas que se marquen como completadas.
- Calcular el tiempo de ejecución de cada función y el tiempo total del programa.

3.2 Estructura del Nodo

```
struct Tarea {  
    string nombre;  
    int prioridad;  
    string estado;    // "pendiente" o "completada"  
    Tarea* siguiente;  
};
```

3.3 Descripción de las Funciones

insertarOrdenado(nombre, prioridad, estado)

Crea un nuevo nodo Tarea y lo inserta en la posición correcta de la lista para mantener el orden descendente de prioridades. Compara la prioridad del nuevo nodo con los nodos existentes, recorriendo la lista hasta encontrar la posición adecuada.

mostrarPendientes()

Recorre la lista completa desde la cabeza hasta el último nodo. Por cada nodo cuyo campo estado sea "pendiente", imprime sus datos en consola. Al final muestra el conteo total de tareas pendientes.

eliminar Completadas()

Recorre la lista en busca de nodos con estado "completada" y los elimina, reconectando correctamente los punteros de los nodos vecinos. Libera la memoria de cada nodo eliminado con delete. Maneja el caso especial de nodos en la cabeza de la lista.

3.4 Análisis de Complejidad Temporal

Función	Complejidad	Justificación
<code>insertarOrdenado()</code>	$O(n)$	Recorre hasta encontrar la posición correcta en la lista ordenada.
<code>mostrarPendientes()</code>	$O(n)$	Recorre todos los nodos para filtrar los pendientes.
<code>eliminarCompletadas()</code>	$O(n)$	Recorre toda la lista y elimina los nodos completados.
<code>mostrarTodas()</code>	$O(n)$	Recorre todos los nodos para imprimirlos.
TOTAL DEL PROGRAMA	$O(n)$	Dominado por las funciones de recorrido lineal.

3.5 Fragmento del Código Implementado

```
void insertarOrdenado(string nombre, int prioridad, string estado) {
    auto inicio = high_resolution_clock::now();
    Tarea* nueva = new Tarea(nombre, prioridad, estado);
    if (cabeza == nullptr || prioridad > cabeza->prioridad) {
        nueva->siguiente = cabeza;
        cabeza = nueva;
    } else {
        Tarea* actual = cabeza;
        while (actual->siguiente != nullptr &&
            actual->siguiente->prioridad >= prioridad) {
            actual = actual->siguiente;
        }
        nueva->siguiente = actual->siguiente;
        actual->siguiente = nueva;
    }
    auto fin = high_resolution_clock::now();
    // Calculo y muestra del tiempo de ejecución
}
```

3.6 Screenshot del Código en ejecución

```
=====
SISTEMA DE GESTION DE TAREAS PRIORIZADAS
Lista Simplemente Enlazada - Equipo Naranja #15
=====

>> Insertando tareas en orden de prioridad...
[insertarOrdenado] Tiempo: 4975 ns
[insertarOrdenado] Tiempo: 324 ns
[insertarOrdenado] Tiempo: 317 ns
[insertarOrdenado] Tiempo: 270 ns
[insertarOrdenado] Tiempo: 3473 ns
[insertarOrdenado] Tiempo: 361 ns
[insertarOrdenado] Tiempo: 360 ns

--- LISTA COMPLETA DE TAREAS ---
Nombre                Prioridad  Estado
-----
Entregar informe final  10        pendiente
Corregir bug critico    9         pendiente
Reunion con el cliente  8         pendiente
Capacitacion de personal 6         completada
Actualizar documentacion 5         completada
Revisar correo          3         completada
Organizar archivos      2         pendiente

--- TAREAS PENDIENTES ---
Nombre                Prioridad  Estado
-----
Entregar informe final  10        pendiente
Corregir bug critico    9         pendiente
Reunion con el cliente  8         pendiente
Organizar archivos      2         pendiente

Total de tareas pendientes: 4
[mostrarPendientes] Tiempo: 59629 ns

[eliminarCompletadas] Se eliminaron 3 tarea(s) completada(s).
[eliminarCompletadas] Tiempo: 7771 ns

--- LISTA COMPLETA DE TAREAS ---
Nombre                Prioridad  Estado
-----
Entregar informe final  10        pendiente
Corregir bug critico    9         pendiente
Reunion con el cliente  8         pendiente
Organizar archivos      2         pendiente

[TIEMPO TOTAL DEL PROGRAMA]: 291 us
=====

...Program finished with exit code 0
Press ENTER to exit console.█
```

4. Ejercicio 2 - Control de Stock Mínimo de Productos



ejercicio2_stock_productos.cpp

4.1 Descripción del Problema

Un minimercado gestiona su inventario mediante una lista enlazada. Cada producto tiene: código, nombre y cantidad disponible en almacén.

El sistema debe:

- a) Insertar los productos con sus datos completos.
- b) Mostrar los productos cuyo stock sea menor a 5 unidades.
- c) Permitir actualizar el stock de un producto dado su código.
- Calcular el tiempo de ejecución de cada función y el tiempo total del programa.

4.2 Estructura del Nodo

```
struct Producto {  
    int codigo;  
    string nombre;  
    int cantidad;  
    Producto* siguiente;  
};
```

4.3 Descripción de las Funciones

insertarProducto(codigo, nombre, cantidad)

Crea un nuevo nodo Producto y lo inserta al final de la lista, recorriendo desde la cabeza hasta encontrar el ultimo nodo (aquel cuyo puntero siguiente es nullptr). Si la lista esta vacía, el nuevo nodo se convierte en la cabeza.

mostrarStockCritico()

Recorre la lista completa evaluando el campo cantidad de cada nodo. Si la cantidad es menor a 5, muestra los datos del producto en consola. Al final indica cuantos productos se encuentran en estado crítico.

actualizarStock(codigo, nuevaCantidad)

Realiza una búsqueda lineal por el campo código. Al encontrar el nodo correspondiente, actualiza su campo cantidad al nuevo valor proporcionado e imprime un mensaje de confirmación indicando el cambio realizado.

4.4 Análisis de Complejidad Temporal

Función	Complejidad	Justificación
<code>insertarProducto()</code>	$O(n)$	Recorre hasta el último nodo para insertar al final.
<code>mostrarStockCritico()</code>	$O(n)$	Recorre todos los nodos para verificar el stock.
<code>actualizarStock()</code>	$O(n)$	Búsqueda lineal por codigo hasta encontrar el producto.

Función	Complejidad	Justificación
mostrarInventario()	O(n)	Recorre y muestra todos los nodos.
TOTAL DEL PROGRAMA	O(n)	Dominado por las funciones de recorrido lineal.

4.5 Fragmento del Código Implementado

```
void actualizarStock(int codigo, int nuevaCantidad) {
    auto inicio = high_resolution_clock::now();
    Producto* actual = cabeza;
    bool encontrado = false;
    while (actual != nullptr) {
        if (actual->codigo == codigo) {
            actual->cantidad = nuevaCantidad;
            encontrado = true;
            break;
        }
        actual = actual->siguiente;
    }
    if (!encontrado)
        cout << "Codigo no encontrado." << endl;
    // Calculo y muestra del tiempo de ejecucion
}
```

4.6 Screenshots del Código en ejecución

```
=====
SISTEMA DE CONTROL DE STOCK - MINIMERCADO
Lista Simplemente Enlazada - Equipo Naranja #15
=====

>> Registrando productos en inventario...
[insertarProducto] Cod 101 - Tiempo: 4128 ns
[insertarProducto] Cod 102 - Tiempo: 241 ns
[insertarProducto] Cod 103 - Tiempo: 155 ns
[insertarProducto] Cod 104 - Tiempo: 158 ns
[insertarProducto] Cod 105 - Tiempo: 159 ns
[insertarProducto] Cod 106 - Tiempo: 1572 ns
[insertarProducto] Cod 107 - Tiempo: 166 ns
[insertarProducto] Cod 108 - Tiempo: 163 ns

--- INVENTARIO COMPLETO ---
Codigo  Nombre                Cantidad
-----
101      Arroz Premium 5lb       12
102      Aceite de Maiz 1L           3
103      Azucar Blanca 2lb           7
104      Sal Refinada 1lb           2
105      Pasta Espaguetti 400g       9
106      Leche Evaporada 400ml       1
107      Habichuelas Negras 1lb      4
108      Jabon de Lavanderia         15

--- PRODUCTOS CON STOCK CRITICO (< 5 unidades) ---
Codigo  Nombre                Cantidad
-----
102      Aceite de Maiz 1L           3
104      Sal Refinada 1lb           2
106      Leche Evaporada 400ml       1
107      Habichuelas Negras 1lb      4

Total productos con stock critico: 4
[mostrarStockCritico] Tiempo: 35412 ns

>> Actualizando stock de productos...

[actualizarStock] Producto "Aceite de Maiz 1L" actualizado: 3 -> 20 unidades.
[actualizarStock] Tiempo: 3067 ns

[actualizarStock] Producto "Sal Refinada 1lb" actualizado: 2 -> 10 unidades.
[actualizarStock] Tiempo: 8390 ns

[actualizarStock] Codigo 999 no encontrado en inventario.
[actualizarStock] Tiempo: 5912 ns
```

```
--- INVENTARIO COMPLETO ---
Codigo  Nombre                Cantidad
-----
101      Arroz Premium 5lb       12
102      Aceite de Maiz 1L           20
103      Azucar Blanca 2lb           7
104      Sal Refinada 1lb           10
105      Pasta Espaguetti 400g       9
106      Leche Evaporada 400ml       1
107      Habichuelas Negras 1lb      4
108      Jabon de Lavanderia         15

--- PRODUCTOS CON STOCK CRITICO (< 5 unidades) ---
Codigo  Nombre                Cantidad
-----
106      Leche Evaporada 400ml       1
107      Habichuelas Negras 1lb      4

Total productos con stock critico: 2
[mostrarStockCritico] Tiempo: 39381 ns

[TIEMPO TOTAL DEL PROGRAMA]: 270 us
=====
```

5. Ejercicio Común - Historial Web con Lista Doblemente Enlazada



ejercicio3_historial_web.cpp

5.1 Descripción del Problema

Un navegador web desea implementar su historial de navegación. Cada nodo en la lista doble representa una página visitada con: URL, fecha y hora de visita.

El sistema debe:

- a) Insertar cada nueva visita al final de la lista.
- b) Permitir al usuario desplazarse hacia adelante o atrás desde la página actual.
- c) Eliminar una URL específica del historial.
- d) Mostrar el historial completo en orden cronológico inverso (ultima a primera visita).
- e) Calcular el tiempo de ejecución de cada función y el tiempo total del programa.

5.2 Estructura del Nodo

```
struct Pagina {
    string url;
    string fechaHora;
    Pagina* siguiente;
    Pagina* anterior;    // Puntero adicional de la lista doble
};
```

5.3 Descripción de las Funciones

insertarVisita(url, fechaHora)

Crea un nuevo nodo Pagina y lo enlaza al final de la lista. Gracias al puntero cola mantenido en la clase, la operación es $O(1)$: el nuevo nodo se conecta al nodo cola actual, se actualiza el puntero anterior del nuevo nodo, y cola pasa a apuntar al nuevo nodo. El puntero paginaActual se mueve a la nueva visita.

navegarAtras() y navegarAdelante()

Mueven el puntero paginaActual una posición hacia atrás (usando el puntero anterior del nodo) o hacia adelante (usando el puntero siguiente). Si ya se está en el primer o último nodo, se notifica al usuario. Ambas operaciones son $O(1)$.

eliminarURL(url)

Realiza una búsqueda lineal por el campo url. Al encontrar el nodo, reconecta correctamente los punteros siguiente y anterior de los nodos vecinos, actualiza cabeza o cola si el nodo eliminado era el primero o el último, y ajusta paginaActual si es necesario. Libera la memoria del nodo eliminado.

mostrarInverso()

Recorre la lista desde el nodo cola hasta la cabeza, utilizando el puntero anterior de cada nodo. Esto permite mostrar el historial en orden cronológico inverso (de la última a la primera visita) en $O(n)$.

5.4 Análisis de Complejidad Temporal

Función	Complejidad	Justificación
<code>insertarVisita()</code>	O(1)	Insertión directa al final usando el puntero cola.
<code>navegarAtras()</code>	O(1)	Acceso directo al nodo anterior mediante puntero.
<code>navegarAdelante()</code>	O(1)	Acceso directo al nodo siguiente mediante puntero.
<code>eliminarURL()</code>	O(n)	Búsqueda lineal por URL en toda la lista.
<code>mostrarInverso()</code>	O(n)	Recorre la lista completa desde cola hasta cabeza.
<code>mostrarCronologico()</code>	O(n)	Recorre la lista desde cabeza hasta cola.
TOTAL DEL PROGRAMA	O(n)	Dominado por las funciones de búsqueda y recorrido.

5.5 Ventaja Clave de la Lista Doble

En este ejercicio, la lista doblemente enlazada ofrece ventajas significativas sobre la lista simple:

- La navegación hacia atrás (`navegarAtras`) es $O(1)$ gracias al puntero anterior, mientras que en una lista simple sería $O(n)$ porque habría que recorrer desde la cabeza.
- La eliminación de un nodo conocido es más eficiente: una vez encontrado, la reconexión de los punteros vecinos se hace en $O(1)$, sin necesidad de rastrear el nodo previo como en la lista simple.
- `mostrarInverso()` es trivial en $O(n)$ recorriendo desde la cola; con lista simple requeriría una estructura auxiliar o recursión.

5.6 Fragmento del Código Implementado

```
void eliminarURL(string url) {
    auto inicio = high_resolution_clock::now();
    Pagina* actual = cabeza;
    while (actual != nullptr) {
        if (actual->url == url) {
            if (actual->anterior != nullptr)
                actual->anterior->siguiente = actual->siguiente;
            else
                cabeza = actual->siguiente;
            if (actual->siguiente != nullptr)
                actual->siguiente->anterior = actual->anterior;
            else
                cola = actual->anterior;
            delete actual;
            tamaño--;
            break;
        }
        actual = actual->siguiente;
    }
    // Calculo y muestra del tiempo de ejecución
}
```

5.7 Screenshots del Código en ejecución

```
=====
SISTEMA DE HISTORIAL WEB
Lista Doblemente Enlazada - Equipo Naranja #15
=====

>> Registrando visitas al historial...
[insertarVisita] "https://www.uapa.edu.do" - Tiempo: 4307 ns
[insertarVisita] "https://www.google.com" - Tiempo: 296 ns
[insertarVisita] "https://www.github.com" - Tiempo: 217 ns
[insertarVisita] "https://www.stackoverflow.com" - Tiempo: 193 ns
[insertarVisita] "https://www.cplusplus.com" - Tiempo: 163 ns
[insertarVisita] "https://www.onlinegdb.com" - Tiempo: 170 ns

--- HISTORIAL (primera a ultima visita) ---
URL                               Fecha y Hora
-----
1. https://www.uapa.edu.do        2025-05-14 08:00
2. https://www.google.com         2025-05-14 08:05
3. https://www.github.com         2025-05-14 08:10
4. https://www.stackoverflow.com   2025-05-14 08:20
5. https://www.cplusplus.com       2025-05-14 08:35
6. https://www.onlinegdb.com       2025-05-14 08:50    << ACTUAL

>> Navegando en el historial...
[navegarAtras] Pagina actual: https://www.cplusplus.com (2025-05-14 08:35)
[navegarAtras] Tiempo: 6415 ns
[navegarAtras] Pagina actual: https://www.stackoverflow.com (2025-05-14 08:20)
[navegarAtras] Tiempo: 1696 ns
[navegarAtras] Pagina actual: https://www.github.com (2025-05-14 08:10)
[navegarAtras] Tiempo: 1675 ns
[navegarAdelante] Pagina actual: https://www.stackoverflow.com (2025-05-14 08:20)
[navegarAdelante] Tiempo: 4888 ns

>> Eliminando URL del historial...
[eliminarURL] "https://www.google.com" eliminada del historial.
[eliminarURL] Tiempo: 5713 ns
[eliminarURL] URL "https://www.yahoo.com" no encontrada.
[eliminarURL] Tiempo: 4492 ns
```

```
--- HISTORIAL (ultima a primera visita) ---
URL                               Fecha y Hora
-----
1. https://www.onlinegdb.com       2025-05-14 08:50
2. https://www.cplusplus.com       2025-05-14 08:35
3. https://www.stackoverflow.com   2025-05-14 08:20
4. https://www.github.com         2025-05-14 08:10
5. https://www.uapa.edu.do        2025-05-14 08:00

Total de paginas en historial: 5
[mostrarInverso] Tiempo: 34565 ns

--- HISTORIAL (primera a ultima visita) ---
URL                               Fecha y Hora
-----
1. https://www.uapa.edu.do        2025-05-14 08:00
2. https://www.github.com         2025-05-14 08:10
3. https://www.stackoverflow.com   2025-05-14 08:20    << ACTUAL
4. https://www.cplusplus.com       2025-05-14 08:35
5. https://www.onlinegdb.com       2025-05-14 08:50

[TIEMPO TOTAL DEL PROGRAMA]: 251 us
=====
```

6. Comparación de Complejidad entre los Ejercicios

Operacion	Ej. 1 (Tareas)	Ej. 2 (Stock)	Ej. Común (Web)
Inserción	$O(n)$ - ordenada	$O(n)$ - al final	$O(1)$ - cola directa
Búsqueda	$O(n)$	$O(n)$	$O(n)$
Eliminación	$O(n)$	N/A	$O(n)$
Navegación atrás	N/A	N/A	$O(1)$
Navegación adelante	N/A	N/A	$O(1)$
Recorrido completo	$O(n)$	$O(n)$	$O(n)$

Observación: El ejercicio de historial web con lista doblemente enlazada destaca por las operaciones de navegación en $O(1)$, lo cual es la ventaja principal de este tipo de estructura frente a la lista simple.

7. Conclusiones

A partir del desarrollo de los tres ejercicios, podemos concluir lo siguiente:

- Las listas simplemente enlazadas son adecuadas para escenarios donde la inserción ordenada, el filtrado y la eliminación selectiva son las operaciones principales, aceptando una complejidad $O(n)$ para la mayoría de operaciones.
- La lista doblemente enlazada es superior cuando se requiere navegación bidireccional o cuando la inserción al final debe ser $O(1)$. El historial web es un caso de uso clásico donde este tipo de estructura ofrece beneficios directos.
- La medición de tiempos con `<chrono>` permite verificar empíricamente la eficiencia de las operaciones. En listas pequeñas, los tiempos son del orden de nanosegundos, lo que confirma la velocidad de las estructuras enlazadas para conjuntos de datos reducidos.
- La implementación bajo el Paradigma Orientado a Objetos (POO) organiza el código en clases con responsabilidades bien definidas, facilitando el mantenimiento, la reutilización y la escalabilidad del sistema.
- El análisis de complejidad Big-O es fundamental para elegir la estructura de datos correcta según el contexto del problema, considerando no solo la funcionalidad sino también el rendimiento esperado a medida que los datos crecen.

8. Referencias

- Carmen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- cplusplus.com. (2024). C++ Reference - `std::chrono`. Recuperado de <https://cplusplus.com/reference/chrono/>
- Universidad Abierta para Adultos (UAPA). (2025). Asignación Escrita II - Estructura de Datos. Santiago de los Caballeros, R.D.